

UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA

Facultatea de Automatică și Calculatoare

Catedra de Calculatoare

Implementarea unei UAL care utilizează numere cu virgulă și negative

Autor: Ilincari Ruxanda

Grupa: 30236

Îndrumător: Mircea Paul Muresan

Data: 14 Ianuarie 2026

Contents

1. Rezumat.....	3
2. Introducere.....	4
3. Fundamentare teoretică	7
3.1 Reprezentarea numerelor în virgulă mobilă	7
3.2 Adunarea și scăderea în virgulă mobilă.....	7
3.3 Înmulțirea numerelor în virgulă mobilă	8
3.4 Împărțirea numerelor în virgulă mobilă	8
3.5 Normalizare, rotunjire și tratarea excepțiilor.....	9
4. Proiectare și implementare.....	10
4.1 Metoda experimentală utilizată	10
4.2 Alegerea soluțiilor de proiectare	10
4.3 Arhitectura generală a sistemului	11
4.4 Descrierea modulelor hardware și a algoritmilor implementați	12
4.5 Detalii de implementare și integrare hardware–software	15
4.6 Manual de utilizare	15
5. Rezultate experimentale	16
5.1 Instrumente și platformă de testare	16
5.2 Verificarea funcțională prin simulare (Vivado)	17
5.3 Testarea sistemului integrat hardware–software (Vitis)	18
5.4 Utilizarea resurselor hardware.....	19
5.5 Discuții și dificultăți întâmpinate	19
5.6 Concluzii asupra rezultatelor experimentale	20
6. Concluzii.....	21
Bibliografie	23

1. Rezumat

În cadrul acestui proiect a fost realizată implementarea unei Unități Aritmetico-Logice capabile să opereze asupra numerelor întregi și a numerelor cu virgulă mobilă, pozitive și negative. Problema abordată a constat în realizarea operațiilor de adunare, scădere, înmulțire și împărțire, respectând regulile standardului IEEE-754 în precizie simplă. Obiectivele principale au fost implementarea operațiilor aritmetice în logică programabilă, integrarea acestora într-un sistem hardware–software și validarea funcționalității. Metoda de rezolvare a implicat utilizarea limbajului VHDL pentru descrierea modulelor hardware în mediul Vivado, dezvoltarea aplicației software de control în limbajul C folosind Vitis și realizarea comunicației între PS și PL prin interfața AXI GPIO. Rezultatele obținute au demonstrat funcționarea corectă a UAL-ului pentru toate operațiile implementate. În concluzie, proiectul a confirmat posibilitatea realizării unei UAL eficiente pentru numere cu virgulă mobilă, evidențiind avantajele utilizării logicii programabile în implementarea operațiilor aritmetice complexe.

2. Introducere

Dezvoltarea sistemelor de calcul moderne este strâns legată de capacitatea acestora de a efectua operații aritmetice complexe într-un timp cât mai scurt și cu un consum redus de resurse. Unitatea Aritmetico-Logică reprezintă nucleul funcțional al oricărui procesor sau accelerator hardware, fiind responsabilă pentru prelucrarea numerică a datelor. În contextul actual, caracterizat de creșterea complexității aplicațiilor din domenii precum procesarea semnalelor, grafica computerizată, inteligența artificială sau sistemele embedded, suportul pentru operații asupra numerelor reale a devenit o cerință esențială.

Tendențele tehnologice recente indică o orientare accentuată către soluții de tip hardware–software co-design, în care sarcinile computaționale intensive sunt delegate logicii hardware specializate, în timp ce partea software asigură flexibilitatea și controlul sistemului. Utilizarea circuitelor programabile de tip FPGA permite implementarea unor unități aritmetice dedicate, adaptate cerințelor aplicației, oferind avantaje semnificative în ceea ce privește performanța și paralelismul față de soluțiile exclusiv software.

Domeniul de studiu abordat în acest proiect este reprezentat de implementarea operațiilor aritmetice în virgulă mobilă la nivel hardware. Numerele cu virgulă mobilă permit reprezentarea unei game largi de valori, atât foarte mici, cât și foarte mari, fiind indispensabile în aplicațiile numerice. Spre deosebire de numerele întregi, reprezentarea numerelor reale presupune utilizarea unui format standardizat care să asigure portabilitate și rezultate consistente. Standardul IEEE-754 s-a impus ca soluție universală pentru reprezentarea numerelor în virgulă mobilă, fiind utilizat în majoritatea arhitecturilor de calcul moderne. Mai multe detalii legate Standardul IEEE-754 pot fi găsite la [1].

Terminologia de bază asociată numerelor în virgulă mobilă include noțiuni precum semn, exponent și mantisă, elemente care alcătuiesc structura unui număr real în format IEEE-754. De asemenea, operațiile aritmetice asupra acestor numere implică etape suplimentare față de aritmetica întregilor, precum alinierea exponenților, normalizarea rezultatului și rotunjirea acestuia. În plus, standardul definește comportamentul pentru situații speciale, cum sunt valorile zero, infinit și NaN, care trebuie tratate corespunzător pentru a asigura corectitudinea calculelor [2].

Problema de rezolvat în cadrul acestui proiect a constat în proiectarea și implementarea unei Unități Aritmetico-Logice capabile să opereze cu numere întregi și numere cu virgulă mobilă, pozitive și negative, realizând operațiile de adunare, scădere, înmulțire și împărțire. Soluția propusă a trebuit să respecte regulile standardului IEEE-754 în precizie simplă (32 de biți) și să fie integrată într-un sistem hardware–software funcțional, capabil să primească date de intrare, să execute operațiile selectate și să furnizeze rezultatele corespunzătoare.

Obiectivele principale ale proiectului au fost:

- înțelegerea modului de reprezentare a numerelor în virgulă mobilă;
- implementarea operațiilor aritmetice fundamentale în logică hardware;
- tratarea corectă a cazurilor speciale definite de standardul IEEE-754;
- integrarea unei soluții hardware cu o aplicație software de control;
- validarea funcționării sistemului prin teste experimentale.

Soluția propusă în acest proiect se bazează pe implementarea operațiilor aritmetice în logica programabilă (PL), folosind limbajul VHDL și mediul de dezvoltare Vivado, în timp ce partea de control și

interfață cu utilizatorul este realizată în C, utilizând mediul Vitis. Comunicarea dintre partea software și partea hardware este realizată prin interfața AXI GPIO, permițând transmiterea operanzilor, selectarea operației și citirea rezultatului. Această abordare este superioară unei implementări exclusiv software prin faptul că permite accelerarea operațiilor aritmetice și o mai bună exploatare a paralelismului hardware [3].

Comparativ cu soluțiile standard oferite de procesoarele generale, implementarea unei UAL dedicate pe FPGA oferă avantajul flexibilității și al controlului detaliat asupra fiecărei etape de calcul. Alegerea algoritmilor utilizați reflectă compromisuri clasice în proiectarea hardware: operațiile de adunare și scădere sunt implementate combinatorial, folosind adunătoare rapide de tip Carry Look-Ahead, în timp ce înmulțirea și împărțirea sunt realizate secvențial, prin algoritmi precum shift-and-add și restoring division.

În secțiunea 3 – Fundamentare teoretică sunt prezentate conceptele teoretice necesare înțelegerii proiectului, incluzând reprezentarea numerelor în virgulă mobilă și algoritmii aritmetici utilizați. Secțiunea 4 – Proiectare și implementare descrie arhitectura sistemului, modulele hardware și software realizate și modul de integrare a acestora. Secțiunea 5 – Rezultate experimentale prezintă testele efectuate și analiza rezultatelor obținute, iar secțiunea 6 – Concluzii sintetizează rezultatele proiectului și evidențiază posibile direcții de dezvoltare viitoare.

3. Fundamentare teoretică

Implementarea unei Unități Aritmetico-Logice capabile să opereze cu numere în virgulă mobilă presupune combinarea conceptelor teoretice legate de reprezentarea numerică standardizată cu metode eficiente de realizare hardware a operațiilor aritmetice.

3.1 Reprezentarea numerelor în virgulă mobilă

Standardul IEEE-754 definește formatul și regulile de calcul pentru numerele reale utilizate în sistemele digitale moderne. În cazul preciziei simple, un număr este reprezentat pe 32 de biți, împărțiți în câmpurile de semn, exponent și mantisă. Această structură permite reprezentarea eficientă a unei game largi de valori, dar impune reguli stricte privind efectuarea operațiilor aritmetice și gestionarea situațiilor speciale [2].

3.2 Adunarea și scăderea în virgulă mobilă

Adunarea și scăderea numerelor în virgulă mobilă presupun compararea exponenților și alinierea mantiselor prin deplasare. După efectuarea operației aritmetice asupra mantiselor, rezultatul este normalizat și rotunjit conform regulilor IEEE-754. Procesul este similar pentru ambele operații, diferența fiind dată de tratarea semnului operandului secund [2].

Din punct de vedere hardware, operațiile asupra mantiselor pot fi efectuate mai multe tipuri de adunătoare. Adunătoarele de tip Ripple-Carry prezintă o structură simplă, dar un timp de propagare al transportului proporțional cu numărul de biți. Adunătoarele Carry Select utilizează mai multe căi de calcul în paralel pentru a reduce întârzierea, iar adunătoarele Carry Look-Ahead calculează anticipat semnalele de transport, obținând timpi de calcul mai reduși. Alegerea tipului de adunător depinde de compromisurile dintre viteză, complexitate și resurse hardware [4].

3.3 Înmulțirea numerelor în virgulă mobilă

Înmulțirea în virgulă mobilă se realizează prin combinarea mai multor etape distincte. Se stabilește semnul rezultatului, exponenții sunt adunați, iar mantisele sunt înmulțite. După calculul produsului mantiselor, rezultatul este normalizat și rotunjit, iar exponentul este ajustat corespunzător pentru a menține formatul standard [2].

Pentru înmulțirea mantiselor, în literatura de specialitate sunt descrise mai multe metode pentru implementarea înmulțirii binare în hardware. O abordare frecvent utilizată este multiplicatorul cu acumulare parțială, bazat pe un sumator pe N biți, care calculează produsul prin acumularea rezultatelor parțiale. O altă metodă este multiplicatorul shift-and-add complet, care utilizează un sumator pe $2N$ biți, fiind bazată pe adunări condiționate și deplasări binare executate la fiecare iterație. În cadrul acestui algoritm, la fiecare ciclu de ceas se verifică bitul cel mai puțin semnificativ al înmulțitorului, iar deînmulțitul este adunat la acumularea parțială doar dacă acest bit are valoarea logică unu. Ambele metode sunt utilizate în implementări secvențiale, diferențele dintre ele fiind legate de complexitatea controlului și de utilizarea resurselor hardware [5].

3.4 Împărțirea numerelor în virgulă mobilă

Operația de împărțire în virgulă mobilă este una dintre cele mai complexe, implicând determinarea semnului rezultatului, scăderea exponenților și împărțirea mantiselor. După obținerea câtului, rezultatul este normalizat și rotunjit, iar exponentul este ajustat conform regulilor IEEE-754 [2].

Pentru implementarea împărțirii mantiselor, literatura de specialitate descrie algoritmul de împărțire cu refacerea restului parțial (restoring division), o metodă clasică bazată pe scăderi succesive și restaurarea restului atunci când rezultatul intermediar devine negativ. Acest algoritm este apreciat pentru claritatea fluxului de control și pentru

robustețea implementării hardware, fiind utilizat frecvent în sisteme secvențiale [6].

O alternativă la algoritmul restoring division este algoritmul de împărțire fără refacerea restului parțial (non-restoring division), care elimină etapa de restaurare prin utilizarea unor operații alternative de adunare și scădere. Deși această metodă poate reduce numărul de operații aritmetice, logica de control este mai complexă [6].

3.5 Normalizare, rotunjire și tratarea excepțiilor

Normalizarea și rotunjirea reprezintă etape esențiale în toate operațiile aritmetice în virgulă mobilă, fiind necesare pentru menținerea rezultatului în formatul standard IEEE-754. Normalizarea constă în ajustarea mantisei și a exponentului astfel încât bitul cel mai semnificativ (ascuns) al mantisei să fie egal cu unu. Rotunjirea este utilizată atunci când precizia rezultatului depășește lungimea mantisei și se realizează, în mod uzual, conform metodei round to nearest, ties to even, care minimizează erorile cumulative în calculele numerice [2].

Standardul IEEE-754 definește, de asemenea, un set de situații excepționale care trebuie tratate explicit, precum operațiile cu zero, infinit sau NaN, precum și cazurile de overflow și underflow. Tratarea coerentă a acestor situații este esențială pentru asigurarea unui comportament determinist și compatibil cu implementările standard [2].

4. Proiectare și implementare

Această secțiune descrie etapele parcurse pentru realizarea Unității Aritmetico-Logice propuse, de la alegerea metodei experimentale și a soluțiilor de proiectare, până la arhitectura generală a sistemului și integrarea componentelor hardware și software. Scopul acestei secțiuni este de a evidenția modul în care fundamentele teoretice prezentate anterior au fost transpuse într-o implementare funcțională.

4.1 Metoda experimentală utilizată

Pentru realizarea proiectului a fost adoptată o abordare de tip hardware–software co-design, utilizând o implementare hardware în logica programabilă (PL) și o implementare software de control pe procesorul din partea de procesare (PS). Modulele hardware au fost descrise folosind limbajul VHDL și sintetizate cu ajutorul mediului de dezvoltare Vivado, în timp ce aplicația software de control și testare a fost realizată în limbajul C, utilizând mediul Vitis.

Această metodă experimentală permite evaluarea directă a comportamentului unității aritmetico-logice într-un sistem real, facilitând atât testarea funcțională, cât și analiza modului de interacțiune dintre componentele hardware și software.

4.2 Alegerea soluțiilor de proiectare

În cadrul proiectării unității aritmetico-logice au fost analizate mai multe soluții posibile pentru implementarea operațiilor aritmetice. Pentru operațiile de adunare și scădere, au fost luate în considerare adunătoarele cu propagarea transportului (Ripple-Carry Adder), adunătoarele de tip Carry Select Adder și adunătoarele Carry Look-Ahead. Dintre acestea, a fost aleasă utilizarea adunătoarelor Carry Look-Ahead, deoarece oferă un timp de propagare redus al transportului, asigurând performanțe superioare față de Ripple-Carry Adder, și au viteza mai mare comparativ cu adunătoarele Carry Select.

Pentru operația de înmulțire au fost analizate două variante: multiplicatorul cu acumulare parțială, care utilizează un sumator pe N biți, și multiplicatorul shift-and-add complet, bazat pe un sumator pe 2N biți. A fost aleasă a doua variantă, deoarece permite calculul direct al produsului mantiselor și oferă un comportament determinist și o implementare clară, chiar dacă implică un consum mai ridicat de resurse hardware.

În cazul operației de împărțire, au fost luați în considerare algoritmi precum non-restoring division și restoring division. A fost selectat algoritmul de împărțire cu refacerea restului parțial datorită clarității fluxului de control și ușurinței de implementare într-o mașină de stări finite, facilitând verificarea și depanarea funcționării.

4.3 Arhitectura generală a sistemului

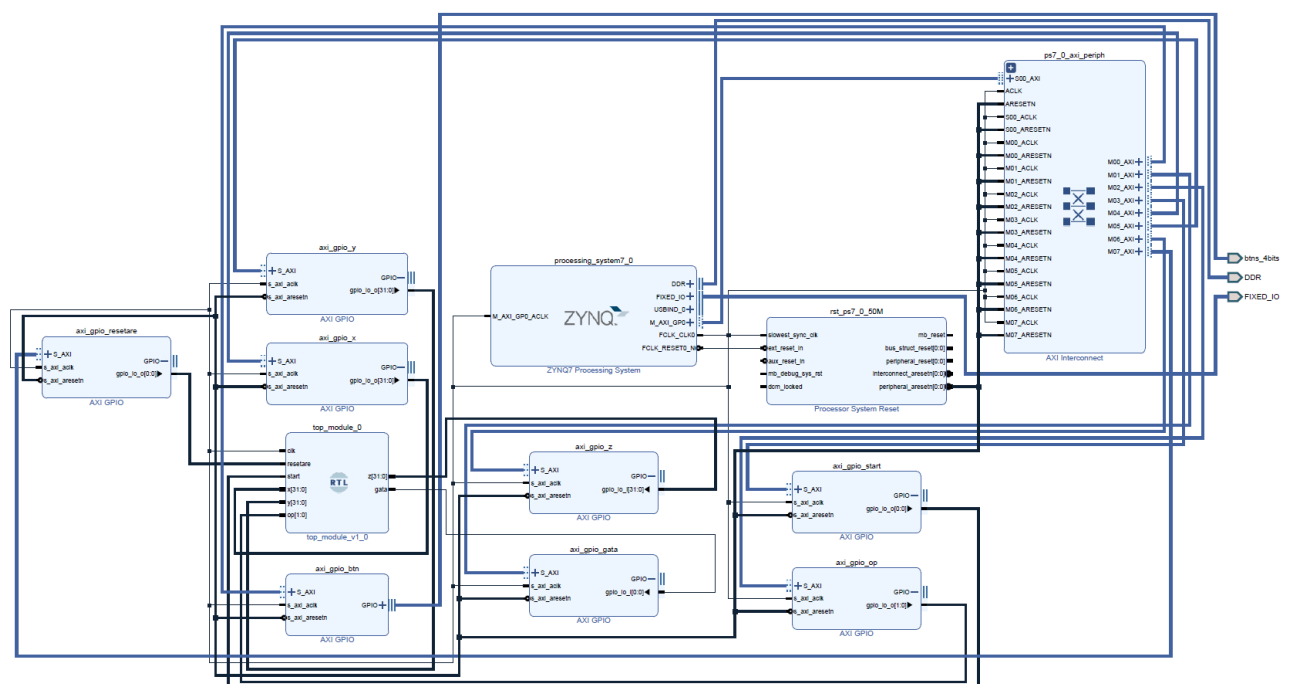


Figura 1: Arhitectura generală a sistemului hardware–software și interconectarea modulelor prin AXI

Arhitectura generală a sistemului este prezentată în Figura 1. Sistemul este implementat pe o platformă Zynq și utilizează o arhitectură hardware–software, în care partea software rulează pe Processing

System (PS), iar operațiile aritmetice sunt realizate în logica programabilă (PL)

Partea hardware include module dedicate pentru fiecare operație aritmetică în virgulă mobilă: adunare, scădere, înmulțire și împărțire. Selecția operației se realizează prin intermediul unui semnal, iar rezultatul este furnizat către partea software. Comunicarea dintre partea software și partea hardware este realizată prin intermediul interfeței AXI GPIO.

Partea software permite introducerea operanzilor, selectarea operației dorite și afișarea rezultatului, asigurând controlul semnalelor de start și monitorizarea semnalului de finalizare a operațiilor secvențiale.

4.4 Descrierea modulelor hardware și a algoritmilor implementați

Implementarea hardware a unității aritmetico-logice este structurată modular, fiecare operație aritmetică fiind realizată de un modul dedicat. Această abordare permite separarea clară a funcționalităților, ușurează verificarea individuală a modulelor și facilitează integrarea într-un sistem hardware–software.

Modulul de adunare în virgulă mobilă (VM_Adder)

Modulul de adunare implementează operația de adunare pentru numere în virgulă mobilă în format IEEE-754, precizie simplă. În prima etapă, operanzii sunt descompuși în câmpurile de semn, exponent și mantisă cu ajutorul unui modul dedicat de descompunere. Mantisele sunt extinse prin adăugarea bitului implicit, iar exponenții sunt aduși într-un format extins pentru a permite comparații corecte.

Operația de adunare presupune alinierea mantiselor prin deplasarea mantisei corespunzătoare operandului cu exponent mai mic. În această etapă sunt calculați și biții de tip guard, round și sticky, necesari pentru rotunjirea corectă a rezultatului. Calculul mantiselor este realizat utilizând adunătoare de tip Carry Look-Ahead pe 24 de biți, asigurând un timp redus de propagare al transportului.

După efectuarea adunării sau scăderii mantiselor, rezultatul este normalizat, exponentul și biții guard, round și sticky fiind ajustați corespunzător. Modulul tratează explicit cazurile speciale definite de standardul IEEE-754, precum operațiile cu zero, infinit sau NaN, precum și situațiile de overflow și underflow. Rezultatul final este recompus în format IEEE-754 și furnizat către modulul de nivel superior.

Modulul de scădere în virgulă mobilă (VM_Substract)

Modulul de scădere are o structură similară cu modulul de adunare, diferența principală constând în tratarea semnului operandului secund. Scăderea este realizată prin inversarea semnului celui de-al doilea operand și aplicarea aceleiași scheme de calcul utilizate pentru adunare.

Etapele de aliniere a mantiselor, calcul al mantiselor folosind adunătoare Carry Look-Ahead, normalizare și rotunjire sunt identice cu cele din modulul de adunare. De asemenea, sunt tratate toate cazurile speciale definite de standard, asigurând compatibilitatea completă cu IEEE-754. Această abordare permite reutilizarea conceptelor și reduce complexitatea generală a implementării.

Modulul de înmulțire în virgulă mobilă (VM_Multiplication)

Modulul de înmulțire implementează operația de înmulțire în virgulă mobilă printr-o abordare secvențială. Este determinat semnul rezultatului, iar exponenții sunt adunați pentru a obține exponentul rezultatului.

Înmulțirea mantiselor este realizată utilizând un multiplicator shift-and-add complet, bazat pe un sumator pe $2N$ biți. Algoritmul este implementat într-un modul secvențial controlat de o mașină de stări finite, care gestionează inițializarea, iterarea și finalizarea operației. La fiecare iterație, se efectuează deplasări ale înmulțitorului și

de înmulțitului, iar adunarea este realizată condiționat, în funcție de bitul curent al înmulțitorului.

După obținerea produsului mantiselor, rezultatul este normalizat, iar exponentul este ajustat corespunzător. Sunt calculați biții necesari pentru rotunjire, iar rezultatul final este recompus în format IEEE-754. Modulul utilizează semnalele de control start și gata pentru sincronizarea cu partea software și cu modulul de nivel superior.

Modulul de împărțire în virgulă mobilă (VM_Division)

Modulul de împărțire realizează operația de împărțire în virgulă mobilă utilizând algoritmul de împărțire cu refacerea restului parțial. Semnul rezultatului este determinat prin XOR-ul semnelor operanzilor, iar exponenții sunt scăzuți pentru a obține exponentul rezultatului.

Împărțirea mantiselor este realizată într-un modul secvențial care implementează algoritmul restoring division, utilizând registre pentru restul parțial și cât, precum și o mașină de stări finite pentru controlul execuției. La fiecare iterație, restul parțial este actualizat, iar bitul corespunzător al câtului este determinat în funcție de semnul rezultatului intermediar. Dacă este necesar, restul este restaurat, conform algoritmului.

După finalizarea procesului iterativ, rezultatul este normalizat și rotunjit, iar exponentul este ajustat. Modulul tratează explicit cazurile speciale, precum împărțirea la zero, operanzii infinit și valorile NaN. Sincronizarea cu partea software este realizată prin semnalele start și gata.

Modulul de nivel superior și integrarea hardware

Toate modulele hardware sunt integrate într-un modul de nivel superior, care realizează selecția operației aritmetice în funcție de un cod de control primit din partea software. Modulul superior asigură rutarea operanzilor către modulele corespunzătoare și transmite

rezultatul către ieșire. În cazul operațiilor secvențiale, semnalul de finalizare este propagat către partea software pentru sincronizare.

4.5 Detalii de implementare și integrare hardware–software

Interfața dintre partea hardware și partea software este realizată prin AXI GPIO, utilizată pentru transmiterea operanzilor, a codului operației și a semnalelor de control. Semnalele de intrare includ operanzii, semnalul de start și codul operației, iar semnalele de ieșire includ rezultatul operației și semnalul de finalizare.

Arhitectura software este structurată în jurul unei aplicații de testare care rulează pe procesorul din PS. Aceasta gestionează inițializarea sistemului, citirea valorilor introduse de utilizator, configurarea semnalelor de control și citirea rezultatului calculului din partea hardware.

4.6 Manual de utilizare

Pentru rularea aplicației este necesar un sistem de operare compatibil cu mediile de dezvoltare Vitis și Vivado. După programarea dispozitivului, utilizatorul introduce valorile operanzilor prin intermediul aplicației software și selectează operația dorită prin apăsarea butonului corespunzător. În cazul operațiilor secvențiale, aplicația așteaptă semnalul de finalizare înainte de afișarea rezultatului.

Aplicația permite efectuarea operațiilor de adunare, scădere, înmulțire și împărțire, afișând rezultatul în format zecimal și asigurând o interfață simplă pentru testarea funcționalității unității aritmetico-logice.

5. Rezultate experimentale

În această secțiune sunt prezentate rezultatele obținute în urma implementării și testării unității aritmetico-logice proiectate. Scopul testelor a fost validarea funcționării corecte a sistemului, atât la nivel de module hardware individuale, cât și la nivel de sistem integrat hardware–software. Rezultatele experimentale confirmă respectarea specificațiilor funcționale și a regulilor standardului IEEE-754 pentru operațiile implementate.

5.1 Instrumente și platformă de testare

Pentru realizarea, implementarea și testarea sistemului au fost utilizate următoarele instrumente și tehnologii:

- limbajul VHDL pentru descrierea modulelor hardware;
- limbajul C pentru dezvoltarea aplicației software de control;
- mediul de dezvoltare Vivado Design Suite (versiunea 2024.1) pentru simularea, sinteza și implementarea designului hardware;
- mediul Vitis Unified Software Platform (versiunea 2024.1) pentru dezvoltarea aplicației software;
- utilitarul hterm pentru afișarea rezultatelor;
- platforma hardware Zynq, care integrează un Processing System (PS) și logică programabilă (PL).

Mediile de dezvoltare Vivado și Vitis au fost utilizate pe un calculator personal cu sistem de operare Windows. Aplicația software de control, scrisă în limbaj C și dezvoltată în Vitis, a fost rulată pe Processing System-ul platformei Zynq și a comunicat cu logica programabilă prin interfața AXI.

5.2 Verificarea funcțională prin simulare (Vivado)

Funcționarea corectă a modulelor hardware a fost verificată inițial prin simulări funcționale realizate în mediul Vivado. Au fost create bancuri de test pentru fiecare operație aritmetică implementată, precum și pentru modulul de top.

Simulările au urmărit:

- corectitudinea rezultatelor obținute pentru adunare, scădere, înmulțire și împărțire;
- propagarea corectă a semnalelor de control;
- comportamentul modulelor secvențiale în raport cu semnalele start și gata;
- tratarea corectă a operanzilor pozitivi și negativi.

Rezultatele obținute în simulare au confirmat funcționarea corectă a fiecărui modul hardware, iar valorile calculate au corespuns rezultatelor așteptate. În Figura 2 sunt atașate rezultatele simulării în vivado.

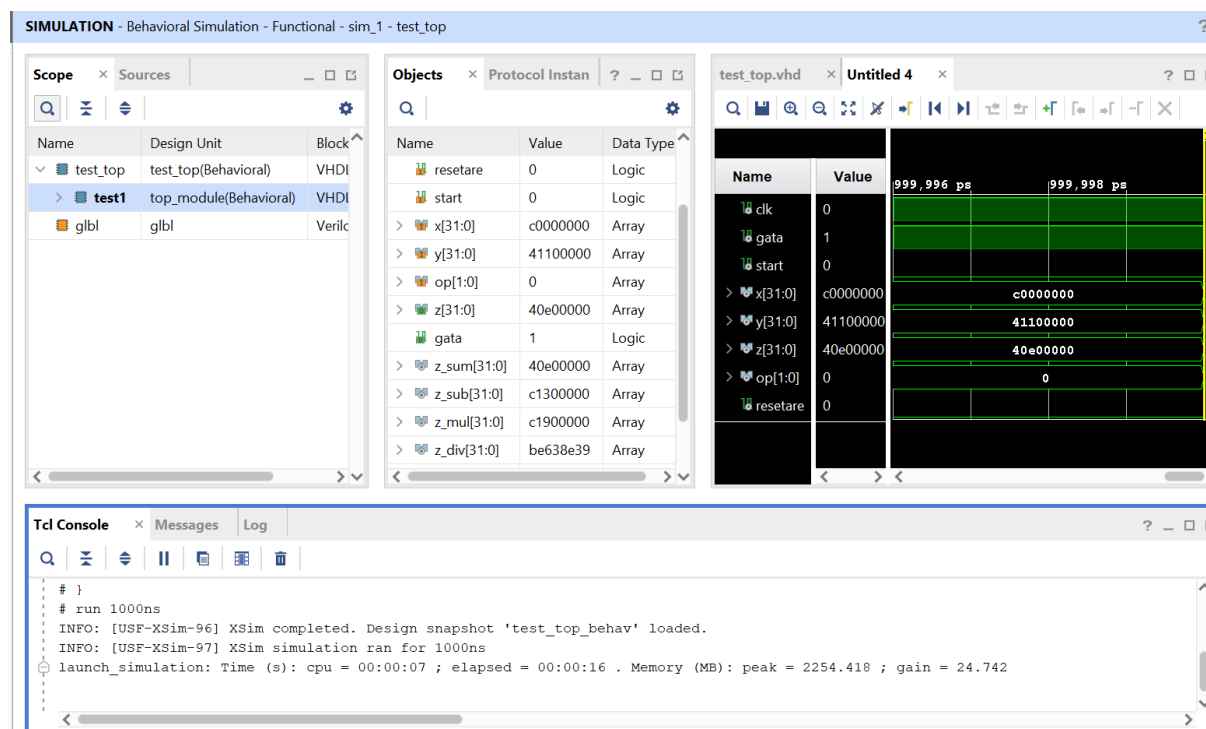


Figura 2: Rezultate preluate din simulator Vivado

5.3 Testarea sistemului integrat hardware–software (Vitis)

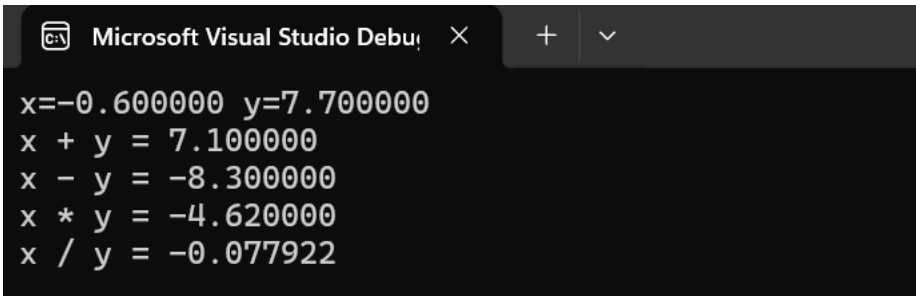
Testarea finală a sistemului a fost realizată pe platforma hardware, utilizând aplicația software dezvoltată în mediul Vitis. Aplicația permite introducerea operanzilor de tip număr real de la tastatură, selectarea operației aritmetice și afișarea rezultatului.

Utilitarul hterm a fost folosit pentru monitorizarea comunicării și pentru vizualizarea rezultatelor obținute.

Pentru validarea rezultatelor obținute pe hardware, valorile calculate de unitatea aritmetico-logică au fost comparate cu rezultatele obținute prin calcule software, utilizând limbajul C. Compararea a confirmat concordanța rezultatelor pentru operațiile testate, demonstrând corectitudinea implementării hardware. În Figura 3 sunt ilustrate rezultatele operațiilor în hterm, iar în Figura 4 sunt ilustrate rezultatele aceluiași calcul în Visual Studio.

```
Introduceti valoarea lui X: \x \n
Am primit x! \x \n
Introduceti valoarea lui Y: \x \n
Am primit y! \x \n
Operanzi setati: X = -0.600000, Y = 7.700000 \x \n
Efectuam adunarea \x \n
Suma Obtinuta este 7.100000 \x \n
Efectuam scaderea \x \n
Diferenta Obtinuta este -8.300000 \x \n
Efectuam inmultirea \x \n
Produsul Obtinut este -4.620000 \x \n
Efectuam impartirea \x \n
Rezultatul obtinut este -0.077922 \x \n
```

Figura 3: Rezultatul operațiilor în hterm



The image shows a screenshot of the Microsoft Visual Studio Debug Console. The window title is 'Microsoft Visual Studio Debug Console'. The console output displays the results of arithmetic operations on variables x and y. The values are: x = -0.600000, y = 7.700000. The operations and their results are: x + y = 7.100000, x - y = -8.300000, x * y = -4.620000, and x / y = -0.077922.

```
x=-0.600000 y=7.700000
x + y = 7.100000
x - y = -8.300000
x * y = -4.620000
x / y = -0.077922
```

Figura 4: Rezultatul operațiilor din Visual Studio

5.4 Utilizarea resurselor hardware

Informațiile privind utilizarea resurselor hardware au fost obținute din rapoartele de implementare generate de mediul Vivado, pentru design-ul implementat pe dispozitivul xc7z010 și sunt prezentate în Tabelul 1.

Tabel 1: Parametri resurse hardware

Blocuri logice	Bistabile	Frecvența maximă
3905	1962	~125 MHz

Analiza utilizării resurselor hardware indică o implementare eficientă a unității aritmetico-logice, cu un consum moderat de blocuri logice și bistabile raportat la complexitatea operațiilor realizate.

Implementarea utilizează 3905 blocuri logice (LUT-uri) și 1962 bistabile, fără a apela la resurse specializate precum blocuri DSP sau memorie BRAM. Această abordare demonstrează că operațiile aritmetice în virgulă mobilă pot fi realizate integral folosind logică generală, asigurând portabilitatea designului și facilitând înțelegerea mecanismelor interne de calcul. Frecvența maximă estimată de aproximativ 125 MHz confirmă faptul că soluția propusă respectă constrângerile de timp și este adecvată pentru aplicații embedded.

5.5 Discuții și dificultăți întâmpinate

În procesul de implementare și testare au fost întâmpinate dificultăți legate de sincronizarea modulelor secvențiale și de coordonarea semnalelor de control între partea hardware și partea software. Aceste probleme au fost rezolvate prin sincronizarea corectă a semnalului gata și propagarea acestuia către modulul de top și ulterior către Processing System.

De asemenea, validarea funcționării pentru cazuri speciale definite de standardul IEEE-754 și rotunjirea au necesitat testare incrementală, atât în simulare, cât și pe platforma hardware. Testarea separată a fiecărui modul a facilitat identificarea și corectarea eventualelor erori.

În urma testelor experimentale s-a observat că tratarea situațiilor de overflow pentru operațiile de înmulțire și împărțire funcționează corect în majoritatea cazurilor uzuale, însă pot apărea limitări pentru anumite cazuri extreme, în special atunci când exponenții rezultatelor depășesc domeniul reprezentabil.

5.6 Concluzii asupra rezultatelor experimentale

Rezultatele experimentale demonstrează că unitatea aritmetico-logica proiectată a fost implementată cu succes și funcționează corect pentru operațiile aritmetice prevăzute. Atât simulările realizate în Vivado, cât și testele efectuate pe platforma hardware, au confirmat validitatea soluției propuse și corectitudinea rezultatelor obținute.

6. Concluzii

În cadrul acestui proiect a fost proiectată și implementată o Unitate Aritmetico-Logică capabilă să efectueze operații de adunare, scădere, înmulțire și împărțire asupra numerelor întregi și a numerelor în virgulă mobilă, pozitive și negative, conform standardului IEEE-754 în precizie simplă. Obiectivul principal al proiectului, acela de a realiza o soluție hardware–software funcțională pentru operații aritmetice complexe, a fost îndeplinit prin integrarea logicii programabile cu o aplicație software de control rulată pe Processing System-ul platformei Zynq.

Implementarea a presupus dezvoltarea modulelor hardware în limbaj VHDL, utilizând adunătoare rapide de tip Carry Look-Ahead pentru operațiile de adunare și scădere, un multiplicator secvențial de tip shift-and-add complet pentru înmulțire și algoritmul de împărțire cu refacerea restului parțial pentru operația de împărțire. Integrarea acestora într-un sistem coerent a fost realizată prin intermediul interfeței AXI, permițând comunicarea eficientă între partea software și logica programabilă. Testele efectuate atât în simulare, cât și pe platforma hardware, au confirmat funcționarea corectă a sistemului pentru cazurile testate.

Contribuțiile originale ale acestui proiect sunt reprezentate de realizarea unei unități aritmetico-logice care implementează operațiile în virgulă mobilă direct în logică hardware fără a utiliza blocurile hardware dedicate pentru operații aritmetice (DSP) sau memorie internă (BRAM), toate calculele fiind implementate folosind logică programabilă generală. Prin această abordare, proiectul evidențiază în mod clar etapele interne ale calculului conform standardului IEEE-754, având un pronunțat caracter didactic. În plus, integrarea modulelor hardware cu o aplicație software de control permite o utilizare flexibilă a sistemului și facilitează verificarea și validarea rezultatelor obținute.

Printre avantajele soluției realizate se numără claritatea arhitecturii, respectarea standardului IEEE-754, portabilitatea designului și posibilitatea extinderii acestuia. Frecvența maximă de funcționare obținută și utilizarea moderată a resurselor hardware demonstrează viabilitatea implementării pentru aplicații embedded. Ca dezavantaje pot fi menționate latența crescută a operațiilor secvențiale de înmulțire și împărțire, precum și limitările observate în tratarea situațiilor de overflow pentru unele cazuri extreme.

Soluția propusă poate fi utilizată în aplicații educaționale, sisteme embedded care necesită suport pentru calcule în virgulă mobilă sau ca bază pentru dezvoltarea unor unități aritmetice personalizate. În ceea ce privește dezvoltările viitoare, proiectul poate fi extins prin optimizarea algoritmilor secvențiali, introducerea unor mecanisme complete de tratare a excepțiilor IEEE-754 sau extinderea suportului către precizie dublă.

Bibliografie

- [1] “IEEE Standard for Floating-Point Arithmetic” in IEEE Std 754-2019 (Revision of IEEE 754-2008) , pp.1-84, 22 July 2019 [Online]. Available: <https://ieeexplore.ieee.org/document/8766229/versions#versions>
- [2] Mircea Paul Muresan, SSC-VM – Cursul 7, Universitatea Tehnică din Cluj-Napoca
- [3] AMD, “What accelerated computing is and why it is important” [Online]. Available: <https://www.amd.com/en/resources/articles/what-is-accelerated-computing-and-why-is-it-important.html>
- [4] Mircea Paul Muresan, SSC Laborator 4, Universitatea Tehnică din Cluj-Napoca – “Implementarea de Sumatoare Hardware Accelerate”
- [5] Mircea Paul Muresan, SSC Laborator 5, Universitatea Tehnică din Cluj-Napoca – “Înmulțire Binara folosind Algoritmul Shift-and-Add pe Zynq”
- [6] Mircea Paul Muresan, SSC Laborator 7, Universitatea Tehnică din Cluj-Napoca – “Utilizarea XADC-urilor și a Împărțirii cu Refacerea Restului Partial pe Zynq”